

Software Architecture and Design Specification

Simple Shell (Command Line Interpreter)

Project: Simple Shell (Command Line Interpreter)

Version: 1.0

Authors: Aarushi Singh (PES2UG24CS015), Akshaya M (PES2UG24CS050), Akshay Anil Kumar (PES2UG24CS049), Adarsh Shivanand Madli (PES2UG24CS028)

Date: 14-09-2026

Status: Final

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Aarushi Singh, Akshaya M, Akshay Anil Kumar, Adarsh Shivanand Madli	Initial draft SAD derived from SRS v1.0

Approvals

Role	Name	Signature/Date
Course Coordinator	Prof. Pavithra	

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Simple Shell, a command-line interpreter (CLI) that parses and executes user commands on a POSIX-compliant Linux system. It translates the functional, non-functional, and security requirements defined in the Simple Shell SRS (v1.0) into a concrete module structure, process design, and interface specification for implementation.

1.2 Scope

This document covers the architecture and design of command input and parsing, built-in command execution, external process execution via fork/exec, foreground and background process management, input/output redirection, pipes, environment variable expansion, bounded command history, and signal handling. It does not cover shell scripting, aliasing, job control (fg/bg/jobs), advanced globbing, networking, or user authentication, all of which are explicitly out of scope in the SRS.

1.3 Audience

Developers, QA/test engineers, the course instructor and teaching assistants, and peer reviewers assessing the design and implementation of the Simple Shell.

1.4 Definitions

CLI, Shell, Built-in command, External command, PATH, PID, stdin/stdout/stderr, Redirection, Pipe, SIGINT, Zombie process, POSIX. Full definitions are given in the SRS, Section 1.4, and are not repeated here.

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for the Simple Shell: a module (component) diagram, module descriptions, the chosen architecture pattern and its rationale, the technology stack, risk analysis, requirements traceability, a threat model, UML sequence diagrams for two representative flows, internal module interface definitions, and UX and error-handling design.

2.2 Related Documents

Software Requirements Specification (SRS) for the Simple Shell, v1.0, 08-09-2026. The Requirements Traceability Matrix (RTM) in SRS Section 8 is the authoritative record of requirement-to-test mapping; Section 3.8 of this document supplements it with requirement-to-module mapping.

3. Architecture

3.1 Goals & Constraints

Goals:

- Correctness: built-in and external commands, redirection, pipes, and variable expansion behave as specified in the SRS.
- Reliability: zero crashes across the functional and negative test suite (SH-NF-002); malformed input is rejected without terminating the shell.
- Responsiveness: the prompt reappears within 1 second of a foreground command completing under normal load (SH-NF-001).
- Portability: builds and runs on at least two POSIX-compliant Linux distributions (SH-NF-005).
- Security: no shell-evaluation of raw input, bounded buffers, and no unintended disclosure of environment state (SH-SR-001 to SH-SR-006).

Constraints:

- Implementation language is C, using only standard POSIX system calls (fork, execvp, wait/waitpid, pipe, dup2, open, chdir, getcwd, signal/sigaction).
- Single-user, single-session, text-only interface; no GUI, no networking, no remote execution.
- No database is used. Command history is maintained in a bounded in-memory list containing at least 50 non-blank commands for the current session. Persistent history across sessions is outside the scope of this version.
- Runs on a standard terminal (tty) with no dedicated hardware.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
End User / Student Developer	Predictable, responsive behaviour; clear error messages; no crashes
System Administrator (representative role)	POSIX-like, predictable file and process management behaviour
Developer	Modularity, testability, and a clean mapping from requirement to module
Course Evaluator / Instructor	Traceability from requirements to design to tests; adherence to the SRS

3.3 Component (UML) Diagram

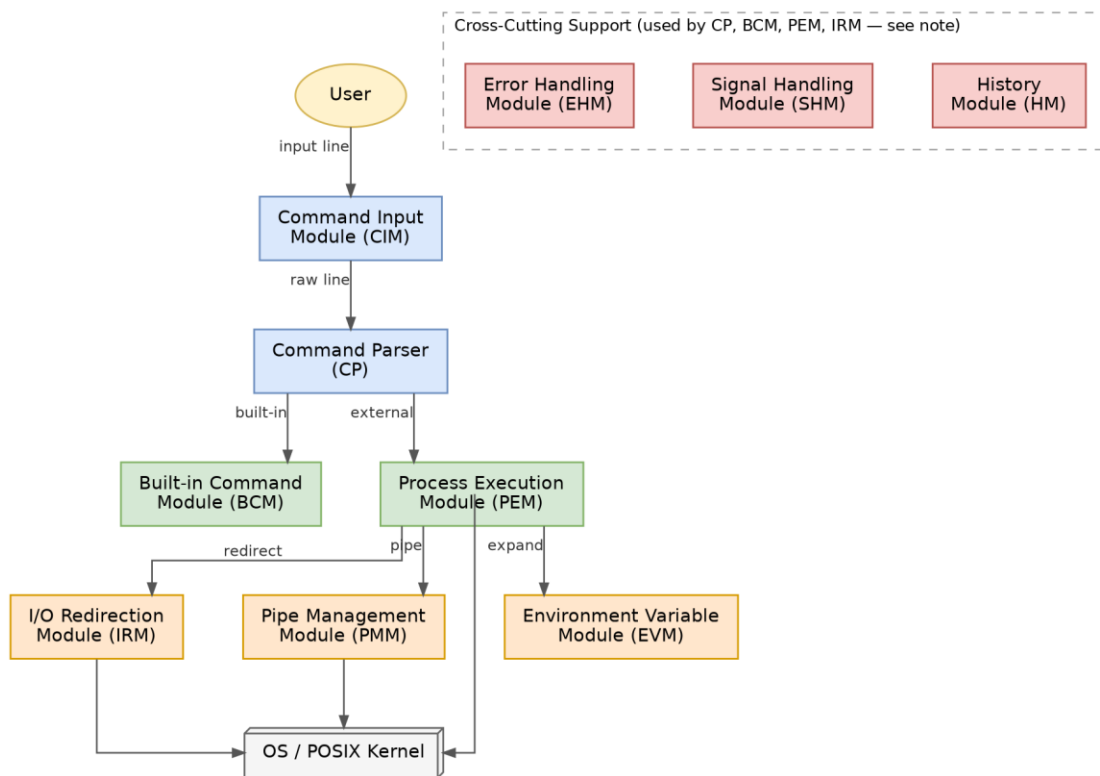


Figure 3.1: Module dependency diagram for the Simple Shell

PEM communicates with the OS kernel via `fork()`, `execvp()`, and `wait()/waitpid()`. The Cross-Cutting Support modules (EHM, SHM, HM) are not tied to a single stage of the pipeline: EHM is called by CP, BCM, PEM, and IRM whenever they detect an error; HM records each successfully parsed command from CP; SHM installs the SIGINT handler used by PEM to manage the running foreground child.

3.4 Component Descriptions

Module	Responsibility
Command Input Module (CIM)	Displays the prompt, reads a line of input into a bounded buffer, and rejects/truncates oversized or malformed byte sequences (SH-F-001, SH-F-002, SH-SR-001, SH-SR-006).

Module	Responsibility
Command Parser (CP)	Tokenizes the input line into a command and arguments, honours single/double quoting, and validates redirection/pipe syntax before execution (SH-F-003, SH-F-004, SH-F-031, SH-SR-005).
Built-in Command Module (BCM)	Implements cd, pwd, echo, help, exit, and history directly inside the shell process (SH-F-006 to SH-F-011).
Process Execution Module (PEM)	Resolves external commands via PATH, forks and execs child processes, waits on foreground children, launches background jobs, and reaps completed children (SH-F-012 to SH-F-018, SH-NF-006).
I/O Redirection Module (IRM)	Implements <, >, and >> by opening files and remapping file descriptors with dup2 before exec (SH-F-019 to SH-F-022, SH-SR-003).
Pipe Management Module (PMM)	Wires the stdout of one command to the stdin of the next using pipe()/dup2() for one or more pipe stages (SH-F-023, SH-F-024).
Environment Variable Module (EVM)	Expands \$VAR references in arguments and propagates the process environment to child processes, without dumping the full environment (SH-F-025, SH-F-026, SH-SR-004).
History Module (HM)	Maintains an in-memory, bounded (≥ 50 -entry) list of non-blank commands (SH-F-027, SH-F-028).
Signal Handling Module (SHM)	Installs a SIGINT handler that terminates only the running foreground child, leaving the shell process alive (SH-F-029).
Error Handling Module (EHM)	Produces a specific, plain-English message for each recognised failure category and is invoked by every other module rather than duplicating error text (SH-F-015, SH-F-030, SH-NF-003).

3.5 Chosen Architecture Pattern and Rationale

The Simple Shell uses a layered, modular-monolith architecture within a single long-lived process: an Input Layer (CIM, CP), a Dispatch Layer that routes a parsed command to either the Built-in Command Module or the Process Execution Module, an Execution Layer (PEM, IRM, PMM, EVM) that carries out the command, and cross-cutting Support Services (HM, SHM, EHM) used by every layer above.

A client-server or microservices style was rejected: the shell has a single local user, no network boundary, and no need for independently deployable services — introducing IPC or service boundaries would add latency and failure modes without any benefit, and would work against the SH-NF-001 responsiveness goal. A layered, single-process design keeps the fork/exec/wait lifecycle simple, maps cleanly onto the module list in the SRS, and keeps each module independently testable and swappable, satisfying SH-NF-004 (one module per source file with a documented interface).

3.6 Technology Stack & Data Stores

- Language: C (C11), compiled with GCC.
- Runtime: glibc on a POSIX-compliant Linux distribution (e.g. Ubuntu 20.04 LTS or later).
- System interfaces: fork, execvp, wait/waitpid, pipe, dup2, open/close, chdir, getcwd, signal/sigaction, getenv/setenv.

- Build tooling: Make/GCC; GDB and Valgrind for debugging and memory-safety verification during development.
- Data storage: no database or external storage service is used. Command history is maintained in a bounded in-memory list containing at least 50 non-blank commands for the current session. Persistent history across sessions is outside the scope of this version.

3.7 Risks & Mitigations

Risk	Mitigation
Buffer overflow from oversized or malformed input	Fixed-size, bounded input and argument buffers with explicit length checks (SH-SR-001, SH-SR-006)
Zombie process accumulation from unreaped background jobs	Completed background processes are periodically reaped using waitpid() with WNOHANG to prevent zombie-process accumulation (SH-F-018, SH-NF-006).
Shell terminates on Ctrl+C along with the running command	When Ctrl+C is pressed, the shell remains active and the signal is handled so that only the running foreground command is terminated (SH-F-029).
Command injection via shell metacharacters	Input is tokenized and executed directly with execvp — never passed through system() or any shell-evaluation call (SH-SR-002)
Malformed pipe/redirection syntax causes a partial, unsafe operation	CP validates full syntax before any file is opened or process is forked (SH-SR-005, SH-F-031)
Unintended disclosure of environment contents	EVM/EHM never dump the full environment block; only explicitly referenced variables are expanded or reported (SH-SR-004)

3.8 Traceability to Requirements

Requirement group	Module(s)
SH-F-001–005 Prompt, input, parsing	CIM, CP
SH-F-006–011 Built-in commands	BCM
SH-F-012–015 External command execution	PEM, EHM
SH-F-016–018 Background execution & reaping	PEM
SH-F-019–022 Redirection	IRM
SH-F-023–024 Pipes	PMM
SH-F-025–026 Environment variables	EVM
SH-F-027–028 Command history	HM
SH-F-029 Signal handling	SHM
SH-F-030–031 Error handling & malformed input	EHM, CP

Requirement group	Module(s)
SH-SR-001–006 Security requirements	CIM, CP, PEM, IRM, EVM, EHM

The full requirement-to-test-case mapping is maintained in the SRS Requirements Traceability Matrix (Section 8) and is not duplicated here.

3.9 Security Architecture

Threat modelling (STRIDE), scoped to a local, single-user, non-networked CLI tool:

- Spoofing: not applicable in the current scope — the shell has no network identity or authentication surface; this is documented as an accepted scope limitation rather than mitigated in code.
- Tampering: malformed or oversized input (unmatched quotes, invalid redirection/pipe syntax, embedded control characters) is validated and rejected by CP/CIM before it can affect execution (SH-SR-001, SH-SR-005, SH-SR-006).
- Information disclosure: the environment block is never dumped in logs, help text, or error messages; only explicitly requested variables are expanded or shown (SH-SR-004).
- Denial of service: bounded input buffers and a fixed-capacity history prevent unbounded memory growth; periodic reaping prevents zombie-process accumulation from exhausting the process table (SH-SR-001, SH-NF-006).
- Elevation of privilege: the shell never bypasses OS file-permission checks for redirection or execution and relies entirely on the underlying kernel's enforcement (SH-SR-003).

4. Design

4.1 Design Overview

The shell runs a read-parse-dispatch-execute loop. CIM displays the prompt and reads a line; CP tokenizes it and validates redirection/pipe syntax; the dispatcher checks whether the command name matches a built-in (routed to BCM) or must be resolved externally (routed to PEM, in cooperation with IRM, PMM, and EVM as needed). SHM and EHM are active throughout the loop rather than at a single stage, and HM records every non-blank line after successful parsing.

4.2 UML Sequence Diagrams

At least two sequence diagrams covering flows specific to this project:

- Flow 1 — External command execution: User -> CIM (input) -> CP (tokenize) -> PEM (PATH lookup) -> fork() -> child: execvp() -> parent: waitpid() -> CIM (redisplay prompt).
- Flow 2 — Piped command execution: User -> CIM -> CP (detect '|') -> PMM (create pipe, fork twice) -> child 1: dup2(stdout, pipe-write) + execvp() -> child 2: dup2(stdin, pipe-read) + execvp() -> parent: close pipe fds, waitpid() on both children -> CIM (redisplay prompt).

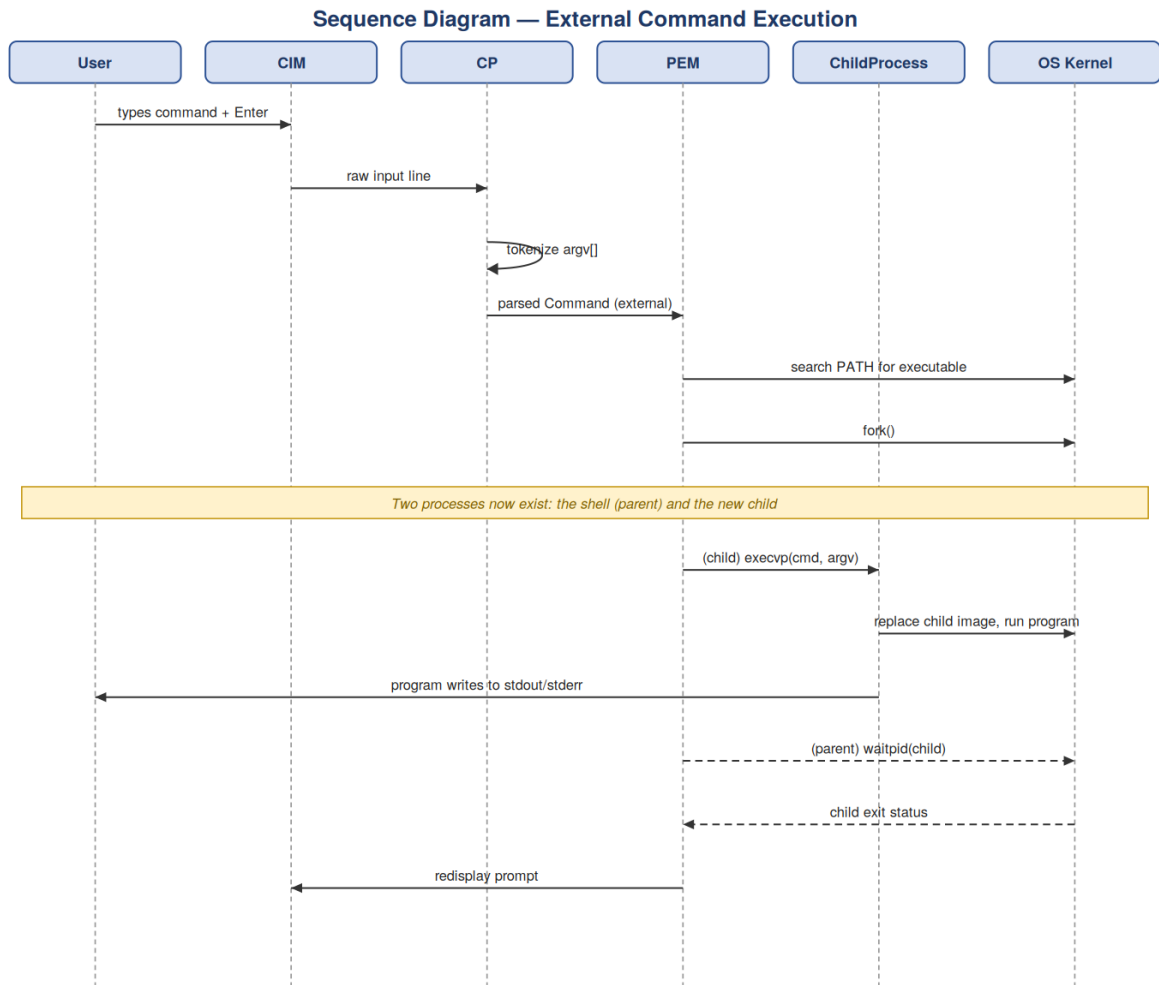


Figure 4.1: Sequence diagram — external command execution

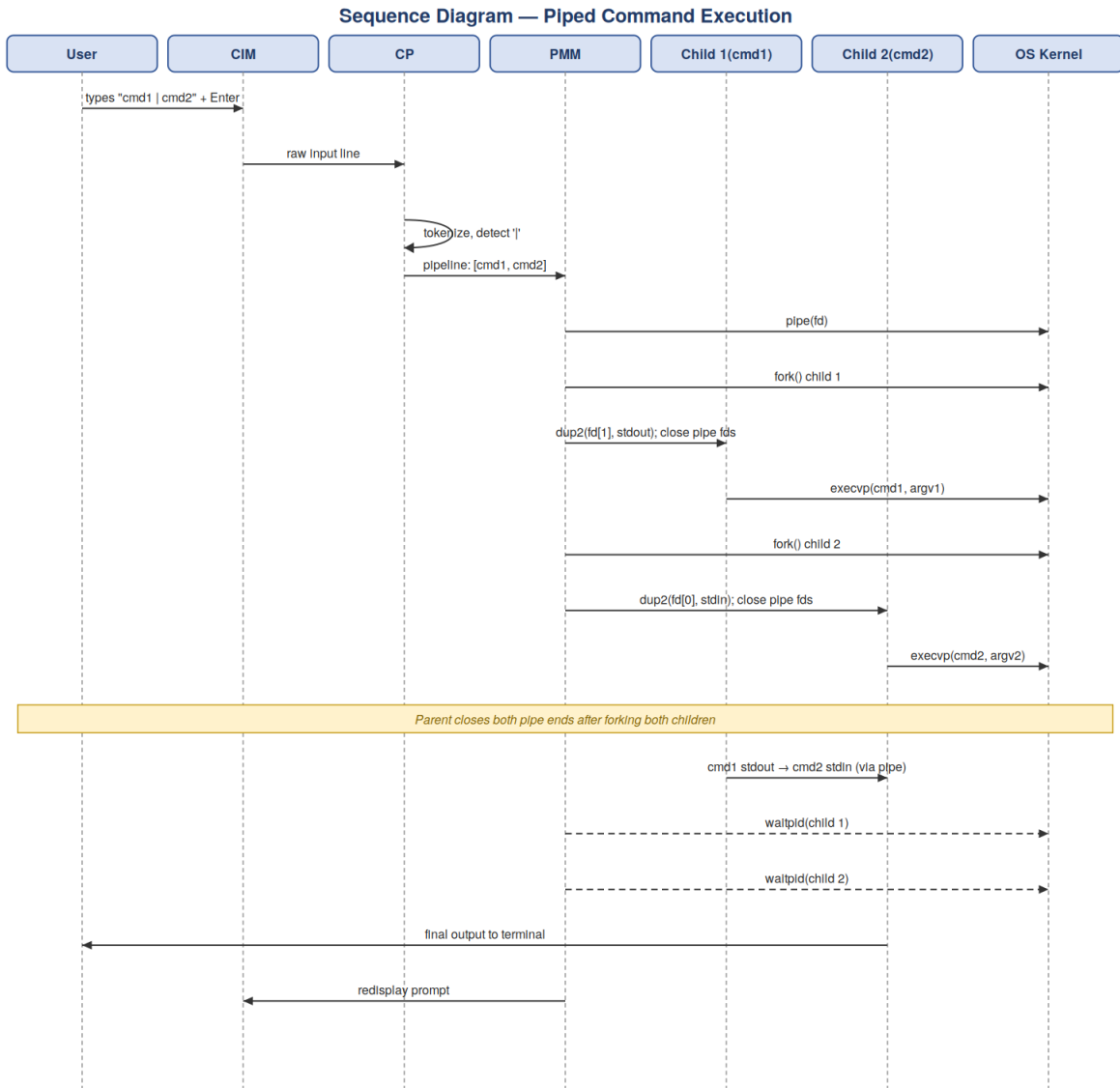


Figure 4.2: Sequence diagram — piped command execution

4.3 API Design

The Simple Shell exposes no network API; "interfaces" here are the internal C function signatures each module exposes to the rest of the shell. Interface definitions for at least two components:

Command Parser (CP)

- Function: `Command *parse_command(const char *line)`
- Input: a raw input line, up to the configured maximum length
- Output: a populated `Command` structure containing `argv[]`, `argc`, `input_file`, `output_file`, `append_flag`, `background_flag`, and pipeline information, or `NULL` if parsing fails or memory allocation is unsuccessful. The caller owns the returned structure and its dynamically allocated fields and must release them using the corresponding cleanup function after execution.

- Errors: sets a shell-wide error code for unmatched quotes, malformed redirection, or a pipe with a missing operand (SH-F-031, SH-SR-005)

Process Execution Module (PEM)

- Function: `int execute_command(Command *cmd)`
- Input: a parsed Command struct from CP
- Output: the exit status of a foreground command. For a background command, execution starts and the function returns immediately while recording the child PID for later reaping.
- Errors: returns a command-not-found or permission-denied status, reported via EHM, without terminating the shell (SH-F-015, SH-F-030)

4.4 Error Handling, Logging & Monitoring

Every recognised failure category — command not found, permission denied, file not found, invalid/malformed syntax — produces a distinct, plain-English message via EHM rather than a raw errno code (SH-F-030, SH-NF-003). No sensitive or full-environment information is written to any message or log (SH-SR-004). As a local CLI tool, the shell has no external monitoring service; the closest equivalents are the exit-status behaviour returned to the parent process and the internal tracking of background-job PIDs used for reaping, both of which are directly observable in the terminal.

4.5 UX Design

The interface is a plain-text prompt (e.g. `shell>`) in a standard terminal emulator. Output uses fixed-width text with no colour or graphical dependency, so it remains legible in any terminal configuration. The help built-in lists every supported command with a one-line description, and history numbers previous commands in entry order, giving the user a simple, discoverable command surface without a GUI.

4.6 Open Issues & Next Steps

- Job control (`fg/bg/jobs`) is explicitly out of scope for this version and is a candidate future enhancement.
- Aliasing and shell scripting/functions are excluded from the current scope.
- Advanced globbing/wildcard expansion beyond simple pattern matching is a candidate future enhancement.
- Persisting history across sessions to a local file is designed for but not mandated by the current requirements.

5. Appendices

5.1 Glossary

CLI, Shell, Built-in command, External command, PATH, PID, stdin/stdout/stderr, Redirection, Pipe, SIGINT, Zombie process, POSIX — as defined in the SRS, Section 1.4.

5.2 References

- IEEE 42010 — Systems and software engineering — Architecture description

- POSIX.1-2017 (IEEE Std 1003.1)
- GNU C Library (glibc) manual
- Simple Shell SRS, v1.0, 08-09-2026

5.3 Tools

- PlantUML / draw.io for UML diagrams
- GCC, GDB, Valgrind
- Git for version control